Assignment 2

Andrew Davidson

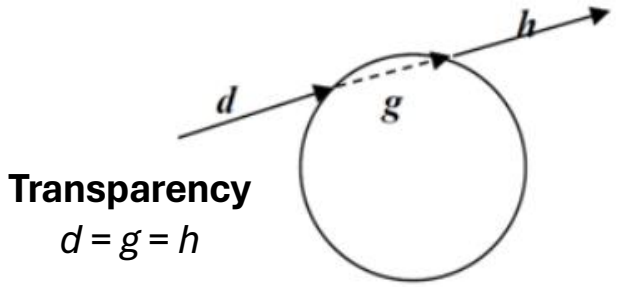
R. Mukundan

Assignment-2

- Due: 11:59pm, **25 May 2026**
  - Late penalty:
  - -0.5 marks for submissions received between 26 May and 29 May (inclusive)
  - -2 marks for submissions received between 30 May and 1 June (inclusive)
- Drop-dead date and time: 1 June 2026, 11:59pm
- Your submission must be based on Lab 7 and 8 code. Implementations using shaders, path tracing, photon mapping etc., not allowed.
- Not a group project. Your submission must represent your own individual work
  - Same generative AI policy as assignment 1
- Students are encouraged to discuss assignment related problems using course forum. However, code segments or any part of your assignment submission should not be posted on Learn.

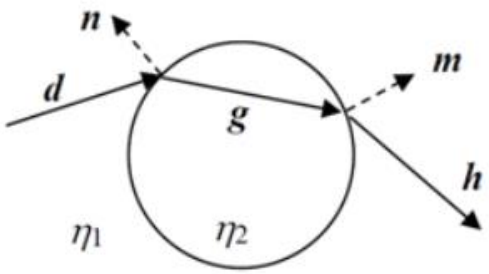
Assignment Specs

- Minimum Reqs (Max. 8 marks out of 20 total)
  - A good spatial arrangement of objects inside a box (or sky box/sphere)
  - Shadows
    - lighter shadows for transparent and refractive objects
  - One planar mirror-like object
  - Chequered pattern or texture on a surface
  - A transparent object. Even though transparency may be treated as a special case of refraction where $\eta_1 = \eta_2$, the implementation of transparency effect should not use the refract() function.



Transparency

d = *g* = *h*



Refraction

g = *refract*(*d*, …)

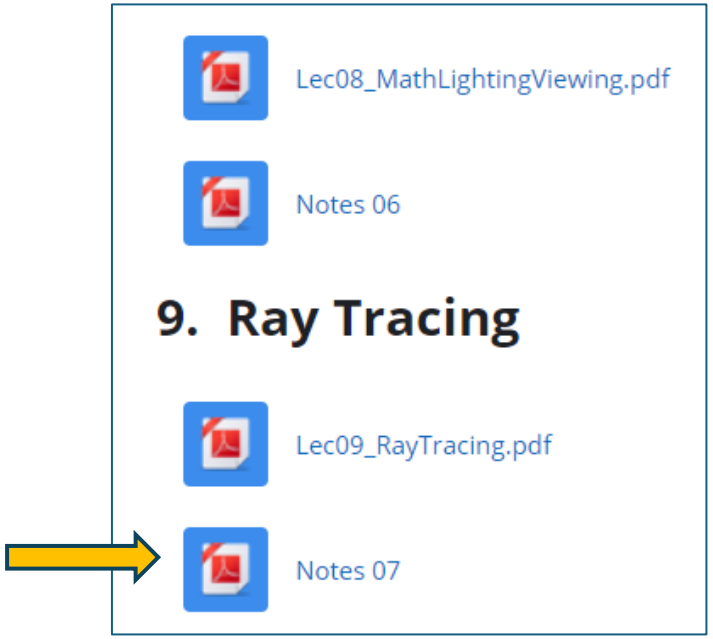
h = *refract*(*g*, …)

Assignment Specs

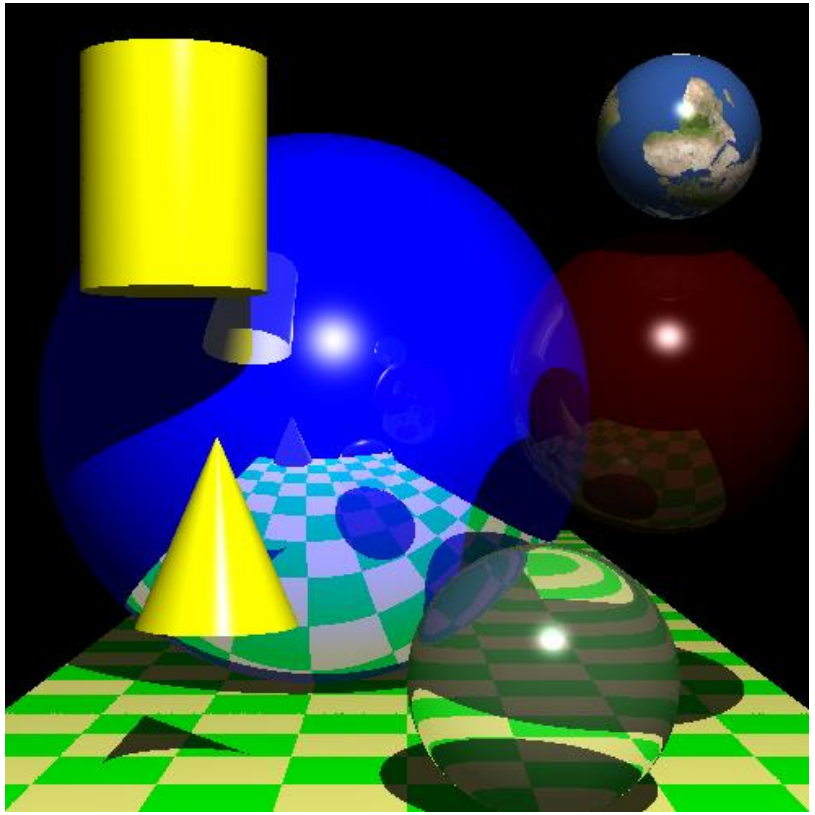
- Extensions (Max. 9 marks out of 20 total)
  - Cone, Cylinder (with cap), Torus
  - Refraction
  - Object-space transformations
  - Anti-aliasing
  - Non-planar object textured using an image
    - E.g., textured sphere, textured cylinder.
  - Spotlight
  - Stochastic sampling
    - Depth of field, soft shadows, or rough surface reflections
  - Multiple light sources: multiple shadows, specular highlights
  - Constructive Solid Geometry

Supplementary Notes

- Information on modelling transparency, multiple light sources and shadows, and spotlights can be found in “**Notes 07**” (Note07_RayTracing.pdf) in lecture material section.



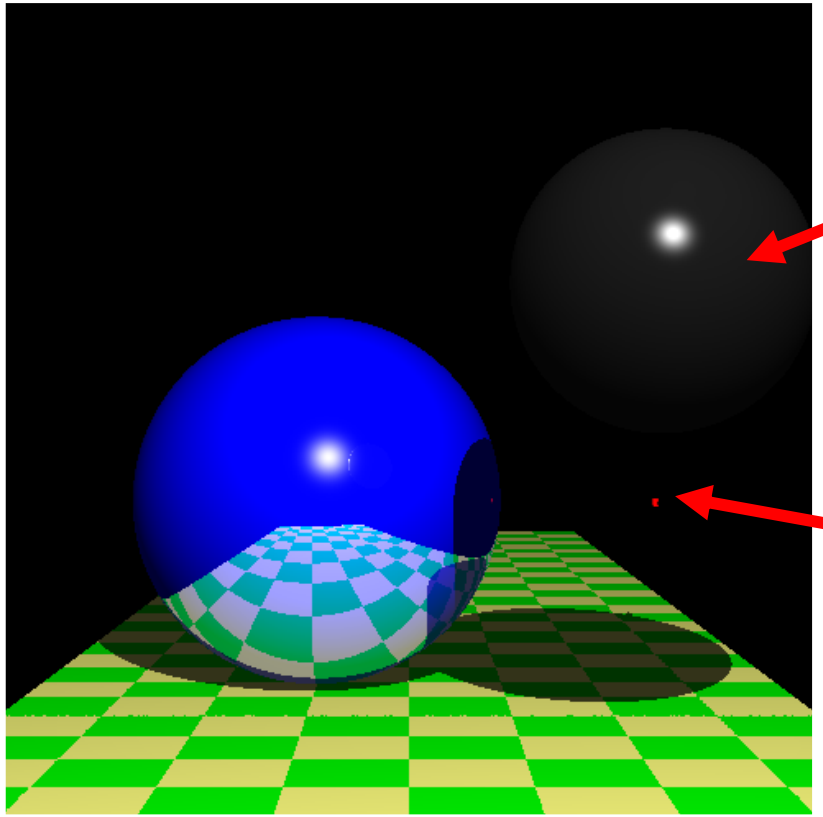
Bad Design



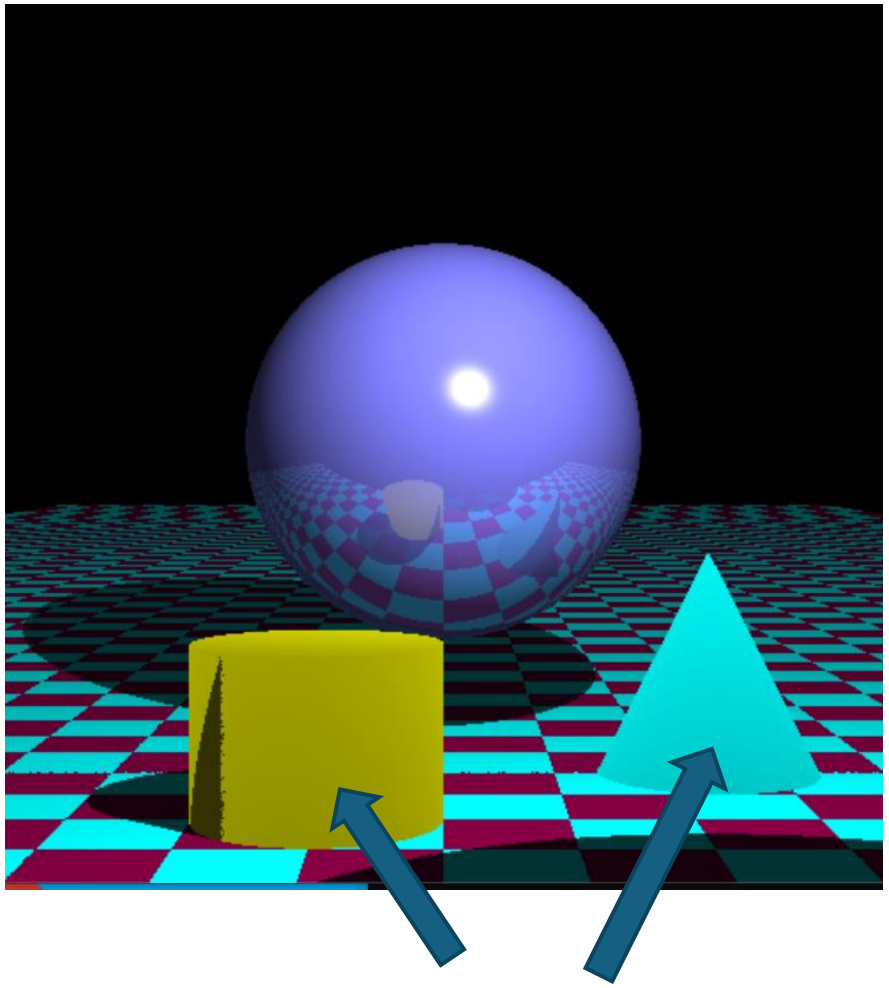
- Random placement of objects
- Objects and features not clearly visible
- Scene clutter
- Incorrect mapping of textures

Bad Design

Marks will not be given to features not clearly visible in the output.



Bad Design

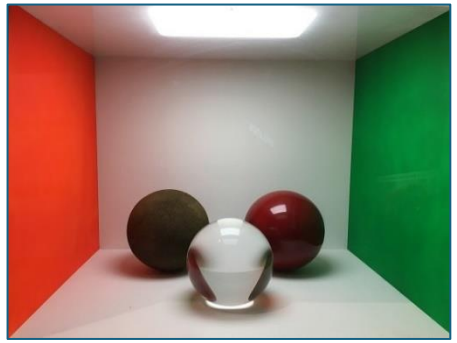


- Improper lighting

Examples of some of the Minimum Requirements

Box

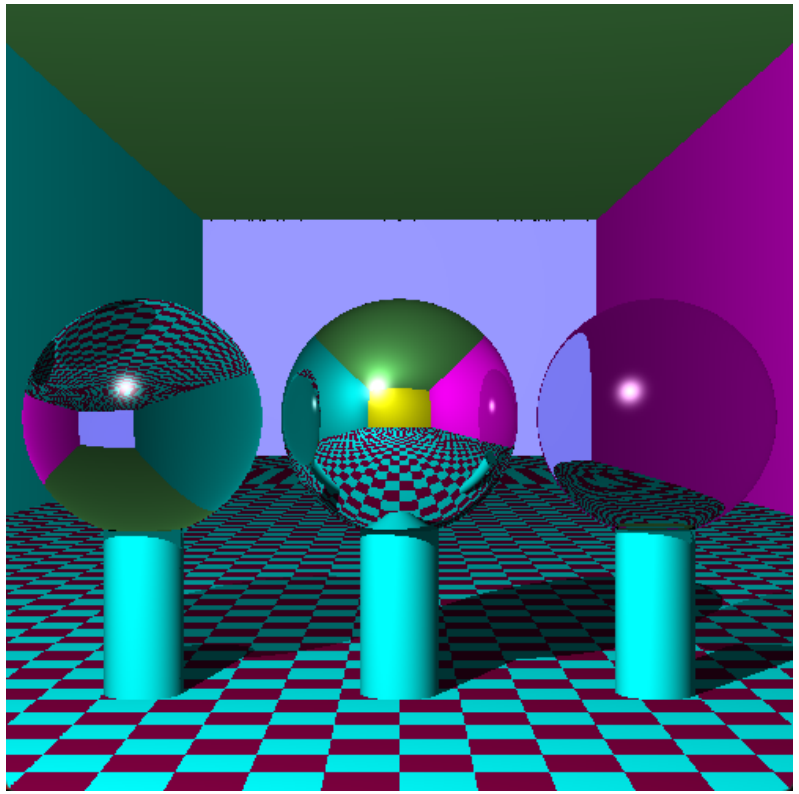
- A box environment is commonly used for testing global illumination algorithms
  - E.g. Cornell Box (Wikipedia)



- 6 axis-aligned planes with different colours OR a textured sky box/sphere
- If using a textured sky box/sphere, lighting calculations must be disabled for the sky box/sphere

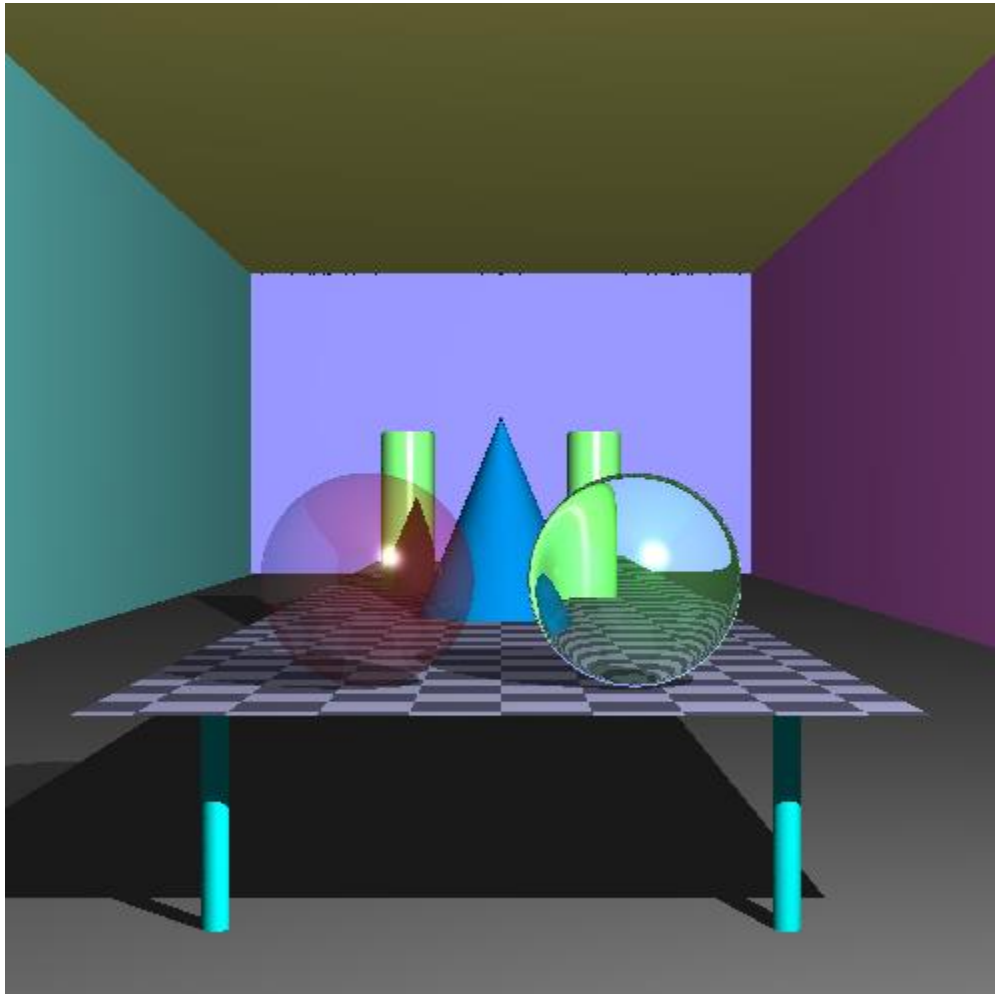
Box: Example

- The example to the right uses 6 planes for the box. Note the reflection of the back plane in yellow colour on the middle sphere
- Spheres: Refractive (η =1.5), Reflective, Refractive (η = 1.005)
- Refractive spheres cast lighter shadows

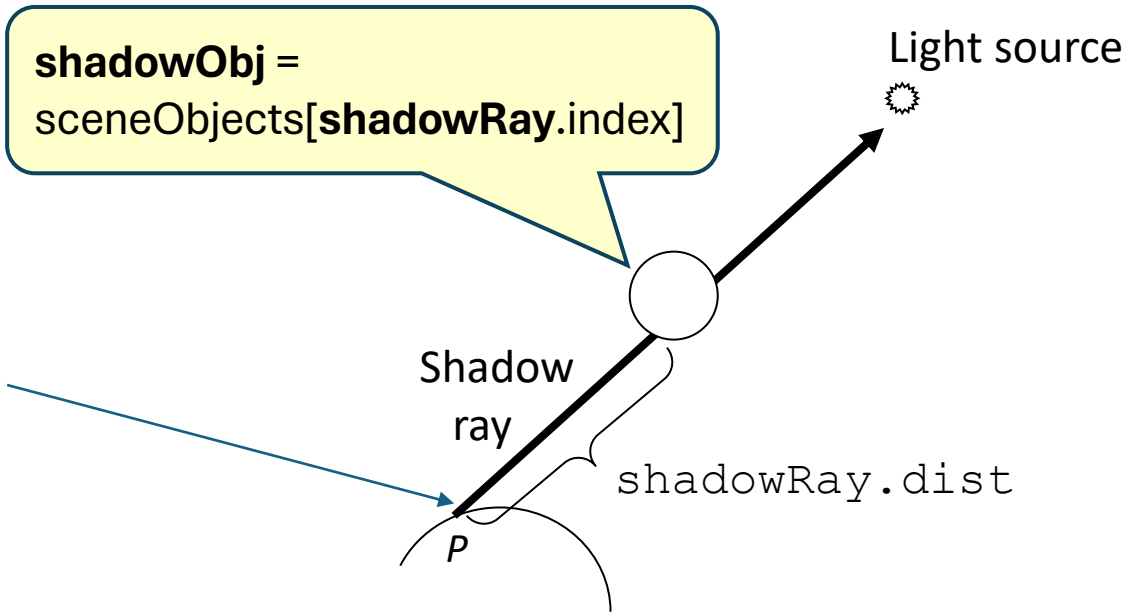


Transparent Object

- Spheres: Transparent, Refractive ($\eta = 1.01$)
- Transparent and refractive spheres cast lighter shadows



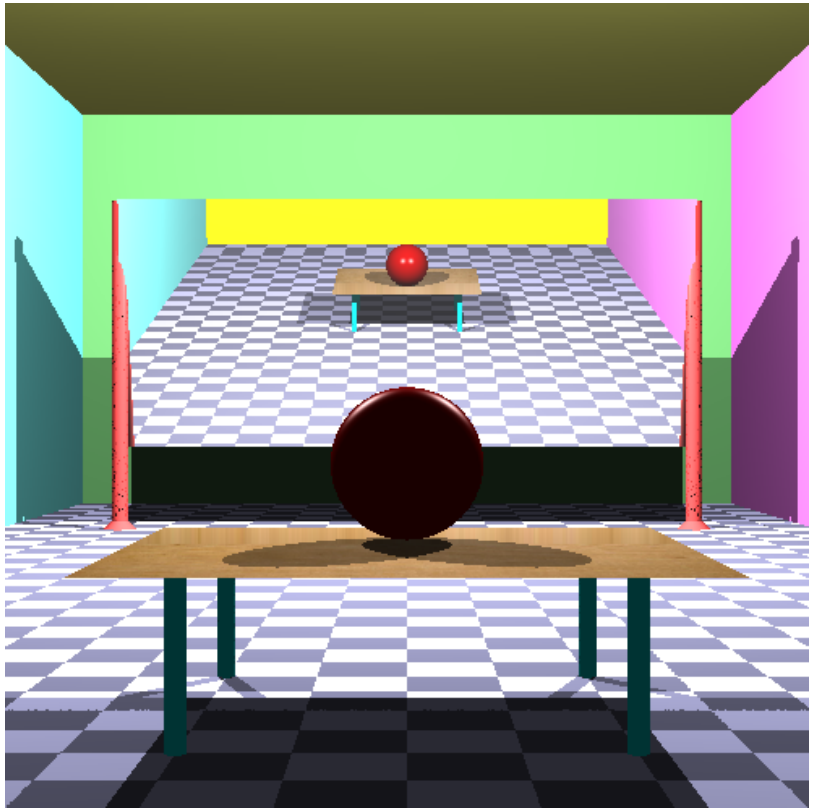
Lighter Shadows

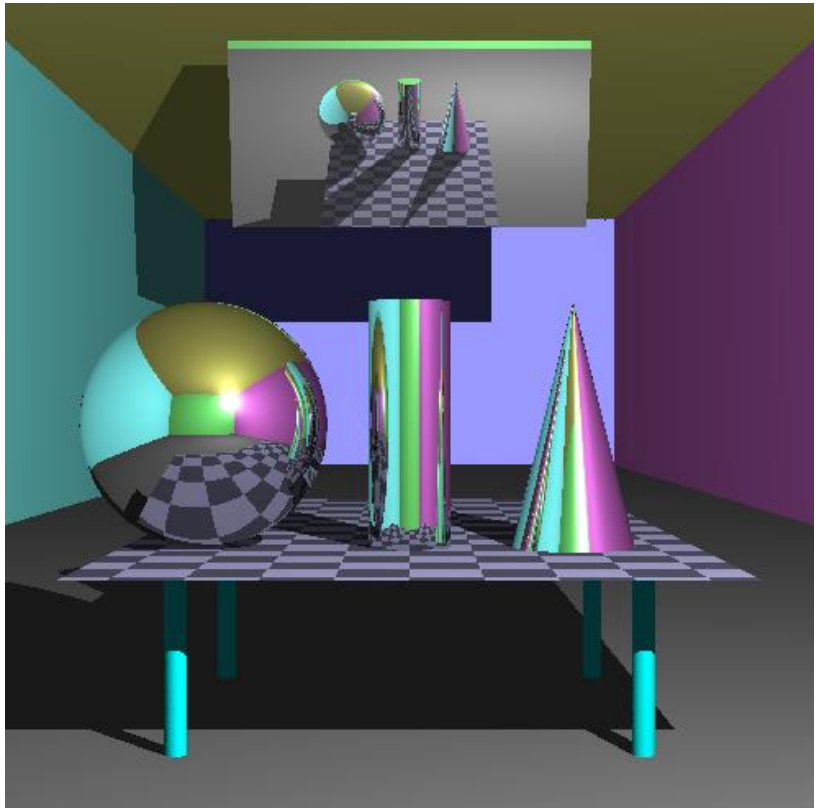


- If `shadowObj` is transparent or refractive, reduce the diffuse and specular term by a proportion instead of deleting them
- The code below could be used to achieve the desired effect, given an object colour `objColour`, and a computed lighting value `lighting` (ambient + diffuse + specular)

```
glm::vec3 ambient = 0.2f * objColor;
glm::vec3 diffSpec = lighting - ambient; //Find the diffuse + specular value
color = ambient + 0.7f*diffSpec;
```

Mirror: Examples

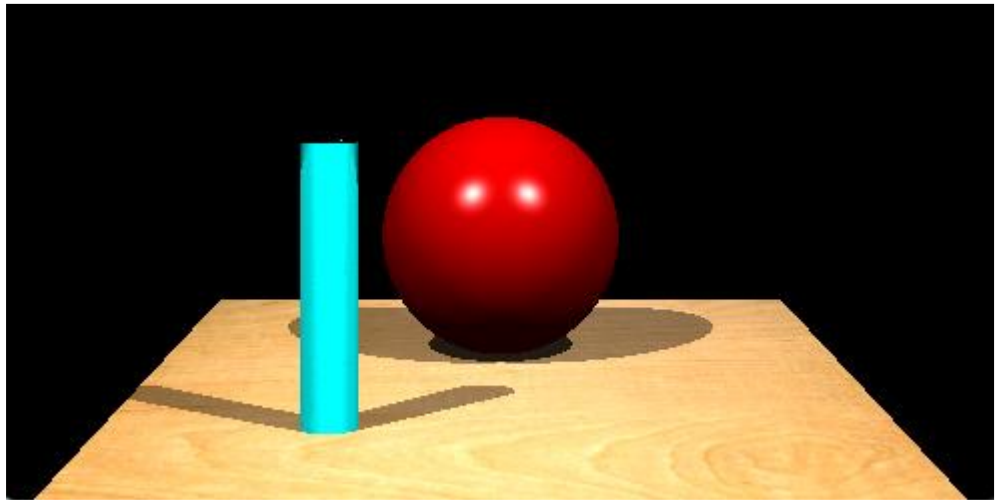




- The first example above contains two light sources

Extra Features

Multiple Lights



- Trace shadow rays to each of the light sources to generate multiple shadows of objects in the scene.
- Multiple specular highlights must be visible on at least one object.

Multiple Light Sources

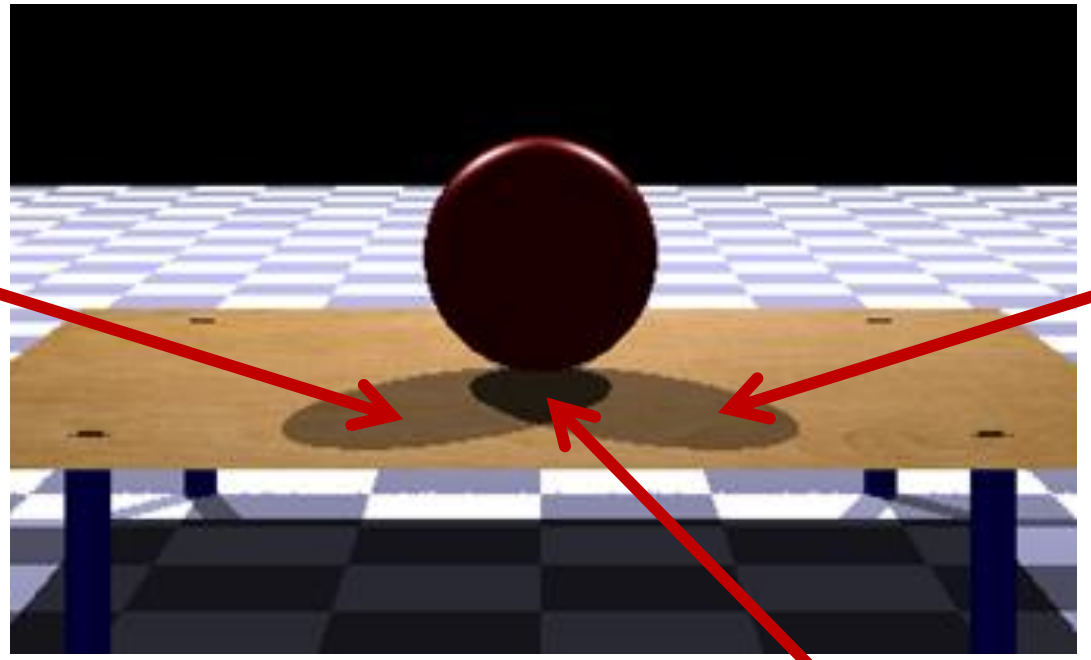
$$\text{colour} = (\text{ambient term}) + (\text{diffuse term})_{L1} + (\text{specular term})_{L1} + (\text{diffuse term})_{L2} + (\text{specular term})_{L2}$$

- Use only **one ambient term**.
  - You may have to modify the function “lighting()” in the SceneObject class.
- Reduce intensity of light sources if required.

Shadows Under Multiple Light Sources

Light source 1

Light source 2



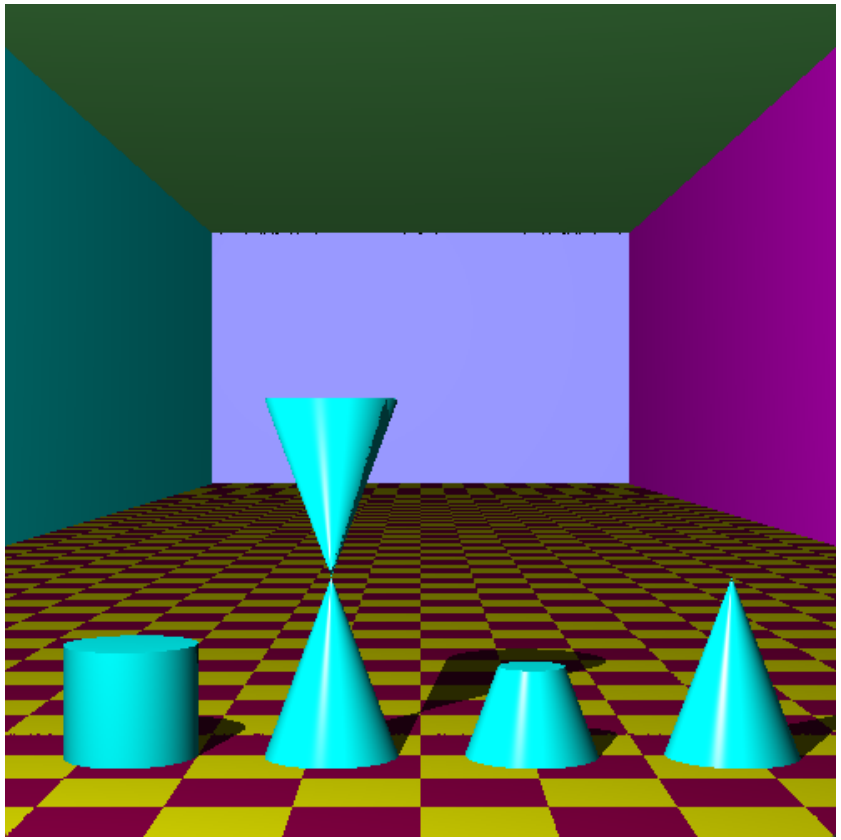
Ambient +
Diffuse$_1$ +
Specular$_1$

Ambient +
Diffuse$_2$ +
Specular$_2$

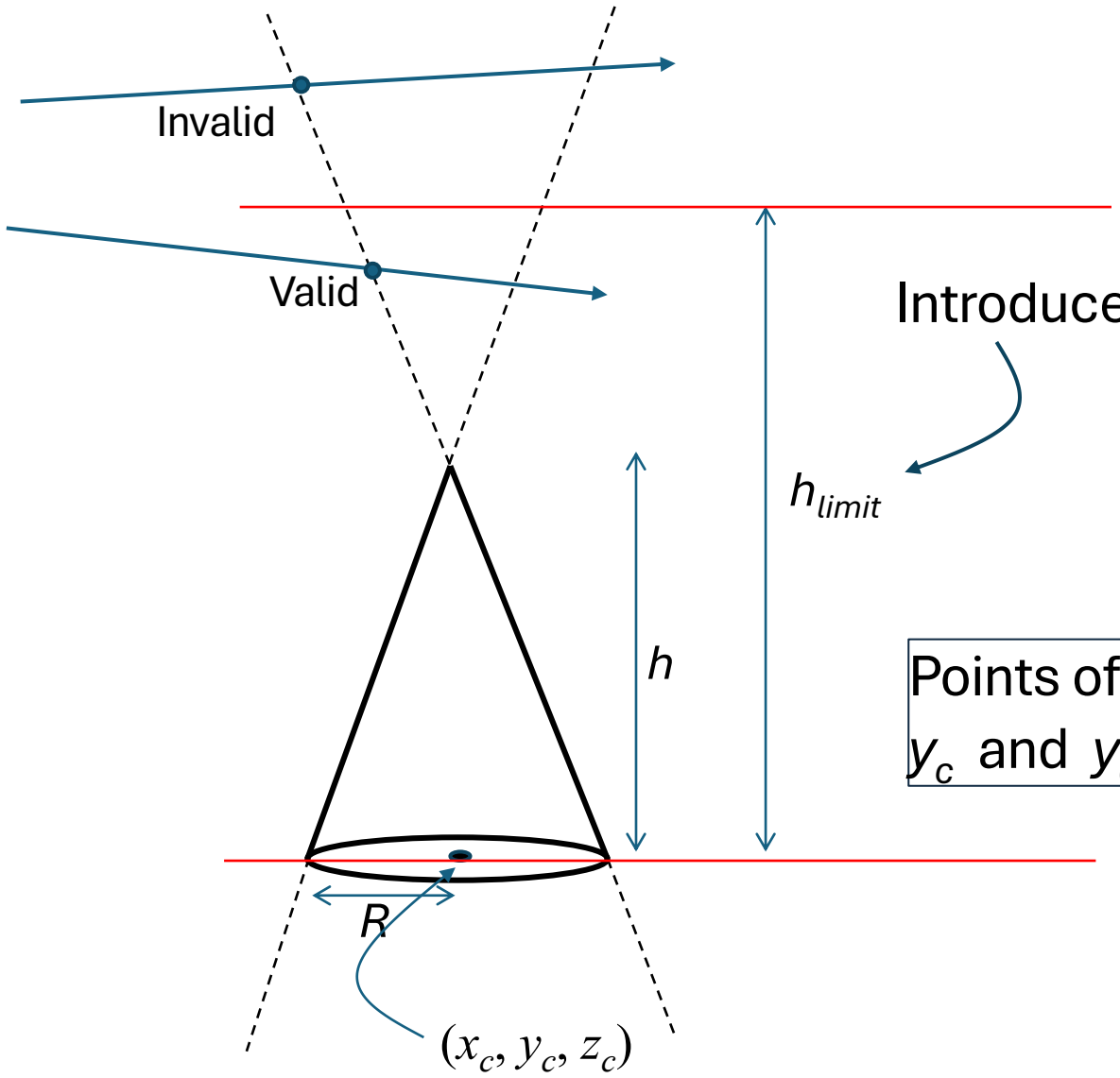
Ambient

Cones

- Double and truncated cones can be generated with slight modifications to the cone intersection method
- In the example to the right, the cylinder and truncated cone have caps



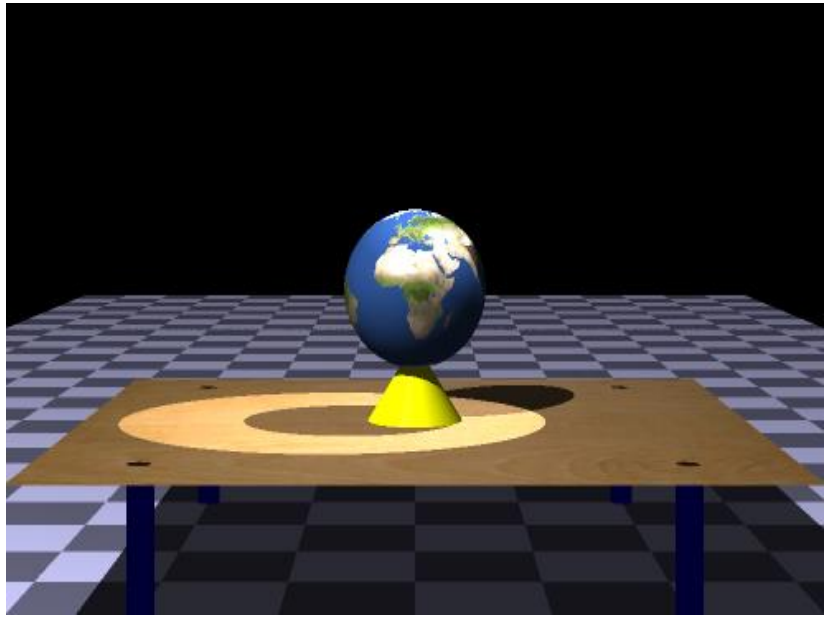
Cones

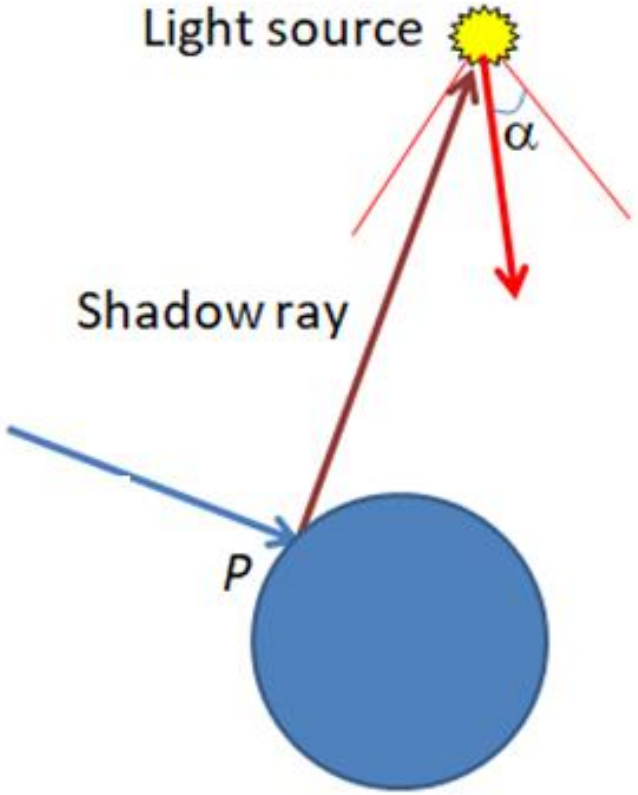


Introduce a new parameter h_{limit}

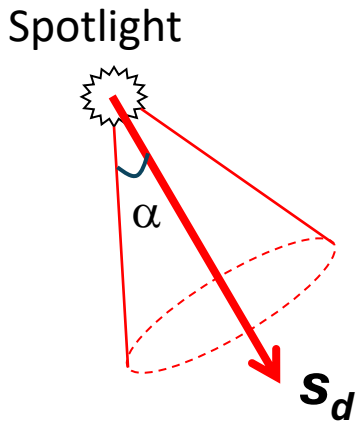
Points of intersection between y_c and y_c+h_{limit} are valid

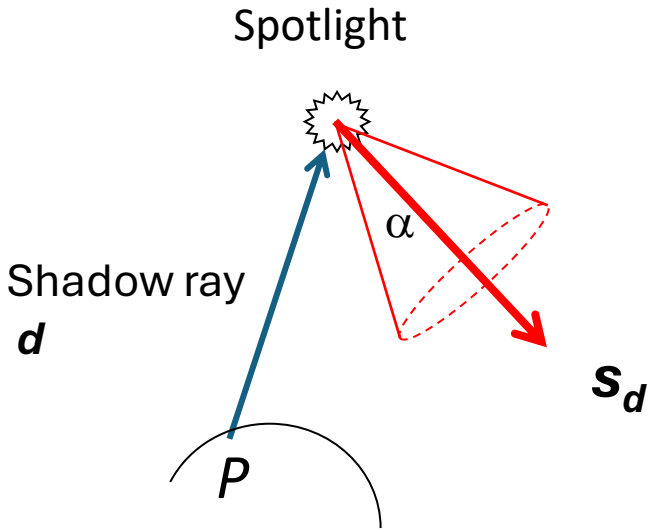
Spotlight





Spotlight

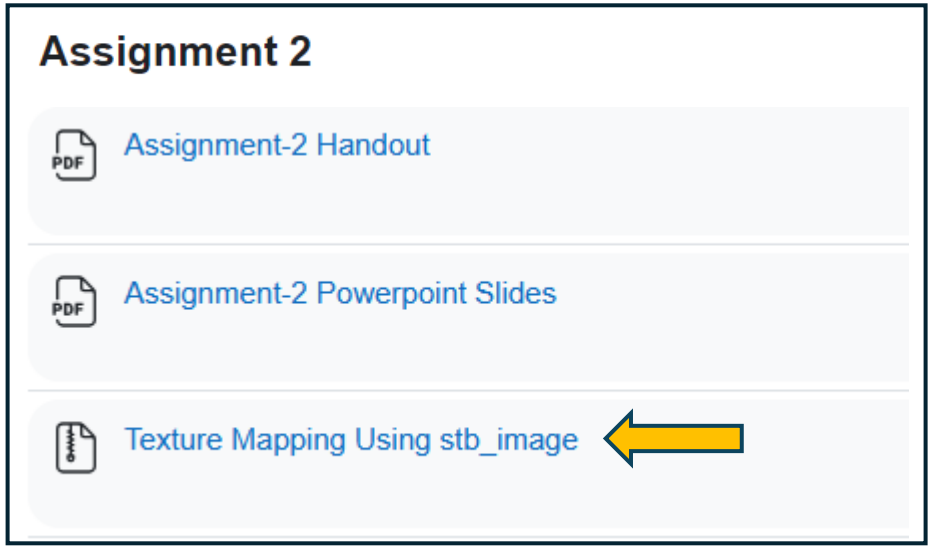




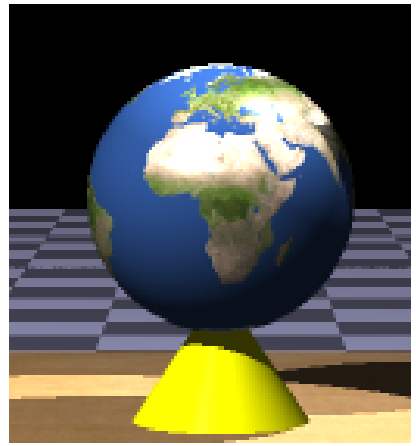
- Requires two additional parameters: a spot direction $\boldsymbol{s_d}$ and a cut-off angle α
- If the point *P* on an object is not in shadow, perform the following additional test:
  - If the angle between the shadow ray d and the spot direction s_d is greater than α, the object is in shadow.
  - Remember to use $-d$ in angle calculation!

Texture Mapping

- Images not loaded by OpenGL functions, and therefore not required to meet “power of 2” requirement for width and height.
- Bitmap Files:
  - 24 bits per pixel (not indexed color)
  - Uncompressed
  - Use files TextureBMP.h, TextureBMP.cpp (Lab08)
- Images in other formats (jpg, png etc.)
  - Can be loaded with image loading libraries e.g. stb_image
  - Use files stb_texture.h, stb_image.h (Assignment section on Learn)

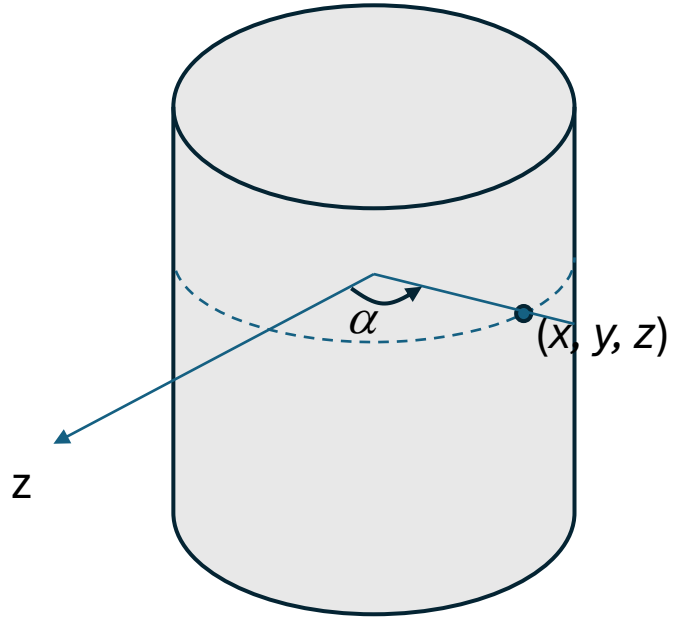


Texturing a Non-Planar Object

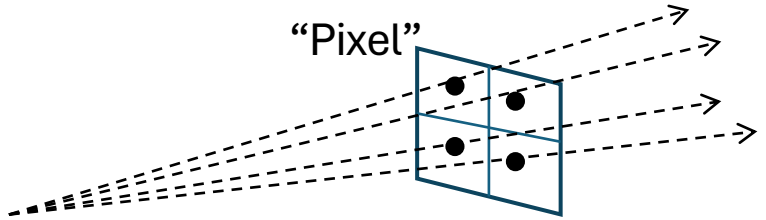


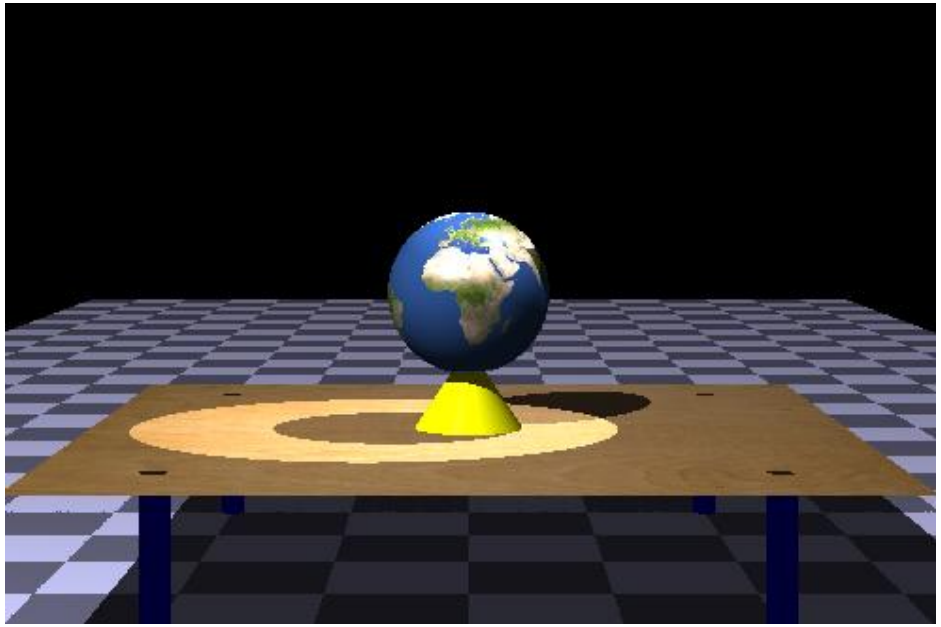
- Sphere: Compute spherical angles α, δ
  - Convert α to texture coordinate *s*
  - Convert δ to texture coordinate *t*
  - Ref: Wikipedia: UV Mapping

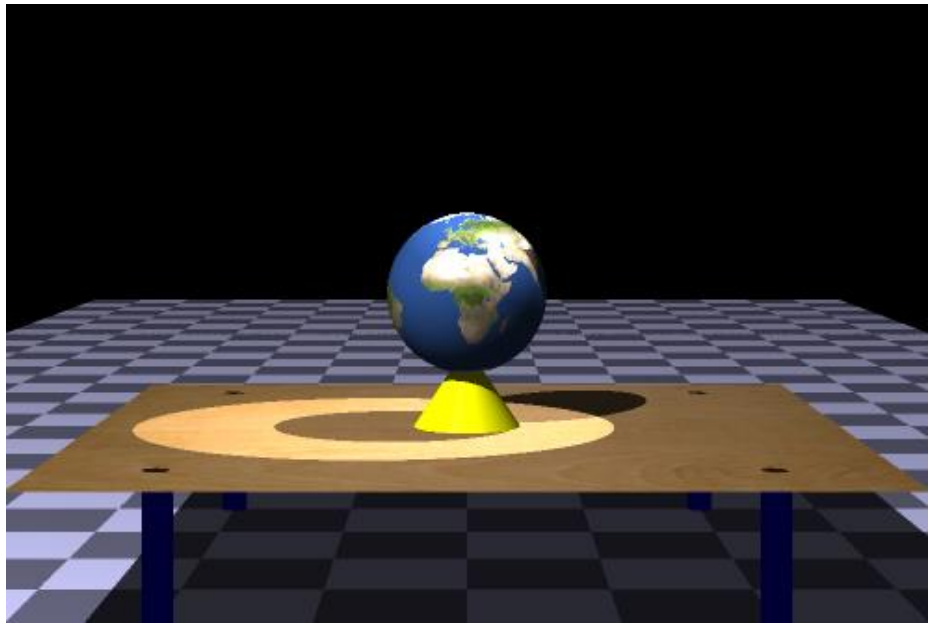
- Cylinder: Compute cylindrical angle α
  - Convert α to texture coordinate *s*
  - Convert *y* to texture coordinate *t*



Anti-Aliasing



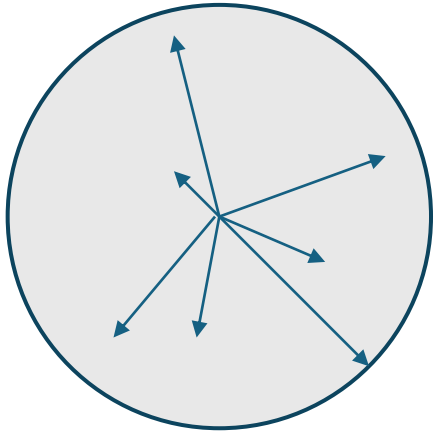




Make sure to include screenshots of outputs with and without anti-aliasing.

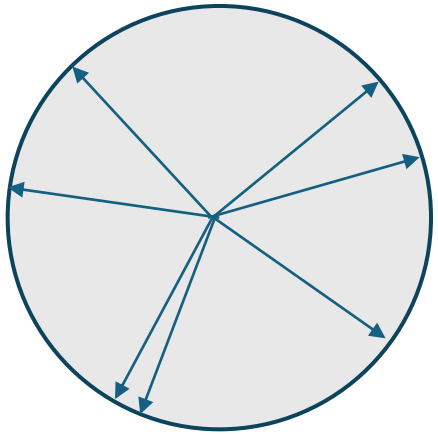
Generating Random Vectors Using GLM

```
#include <glm/gtc/random.hpp>
```



```
glm::vec2 rvec = glm::diskRand(radius);   //2D
```

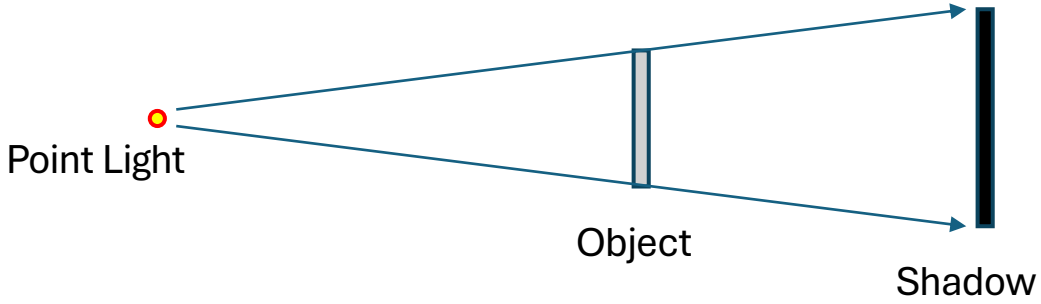
```
glm::vec3 rvec = glm::ballRand(radius);   //3D
```

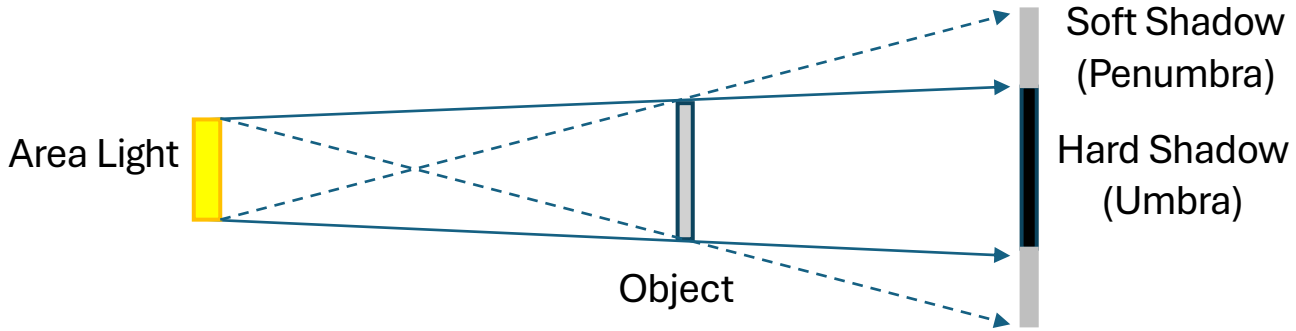


```
glm::vec2 rvec = glm::circularRand(radius);   //2D
```

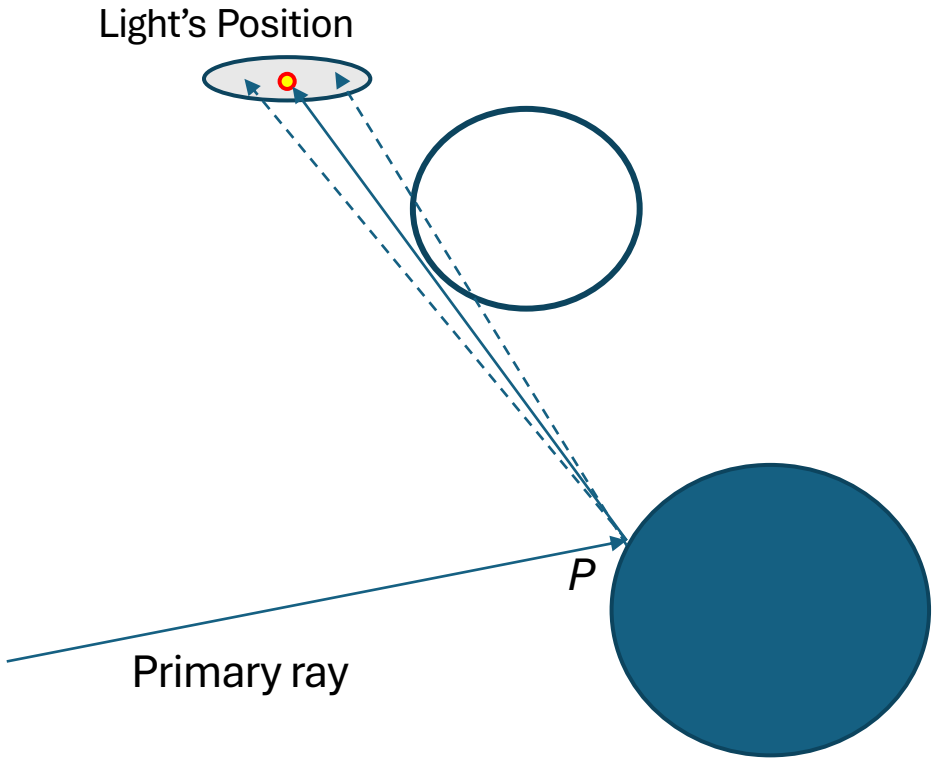
```
glm::vec3 rvec = glm::sphericalRand(radius);   //3D
```

Area Lights and Soft Shadows



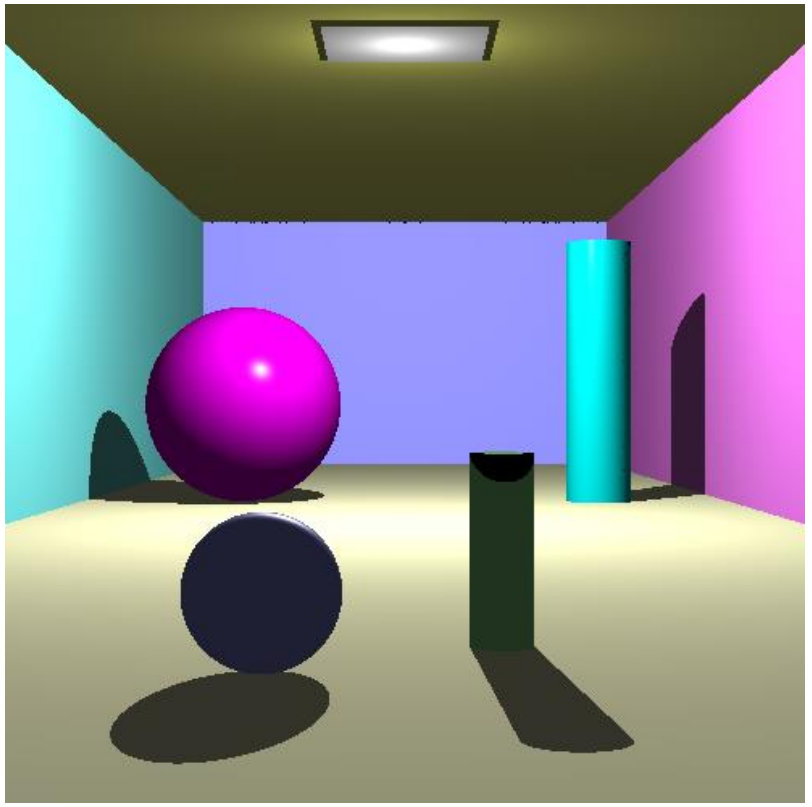


Area Lights

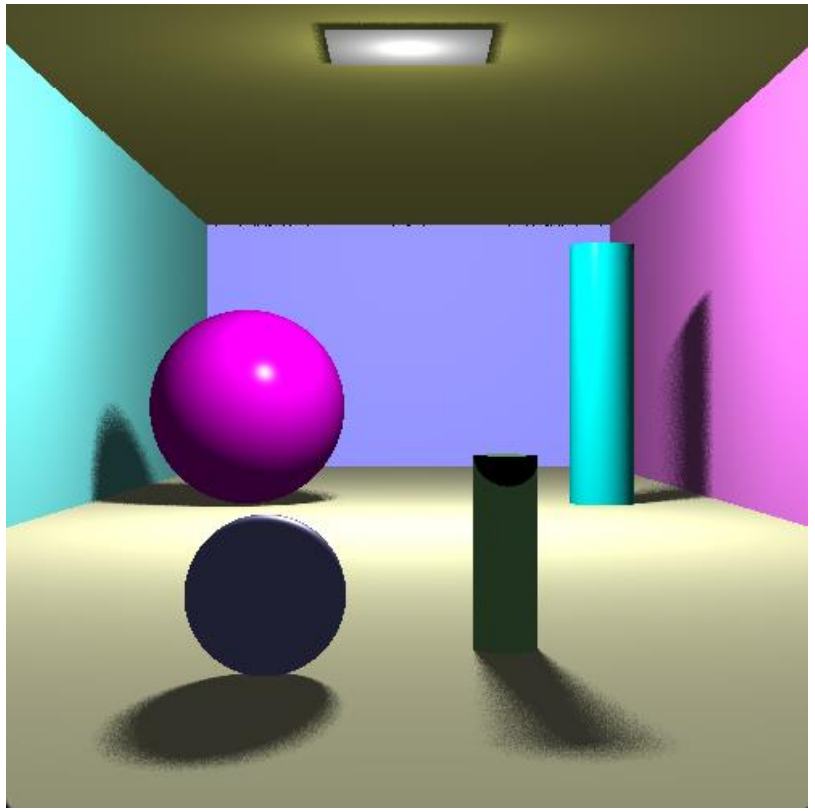


- Define a small circular region around a light source.
- Sample this area using a set of randomly distributed points, and generate multiple shadow rays to this region.
- Take the average of colour values obtained from each shadow ray.

Soft Shadows



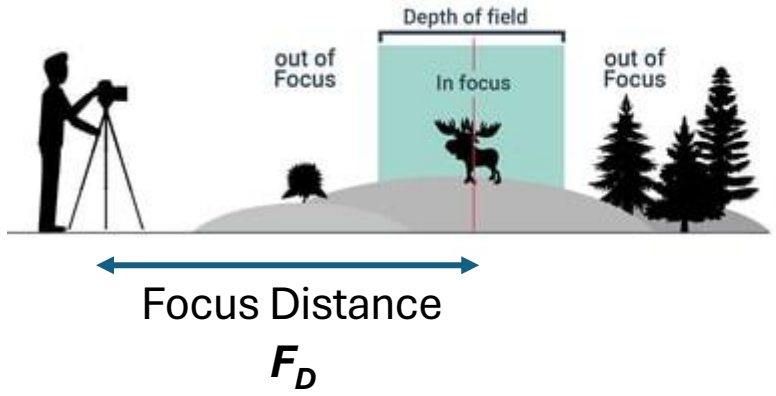
Shadows Using Point Light Source



Shadows Using Area Light Source

Depth of Field

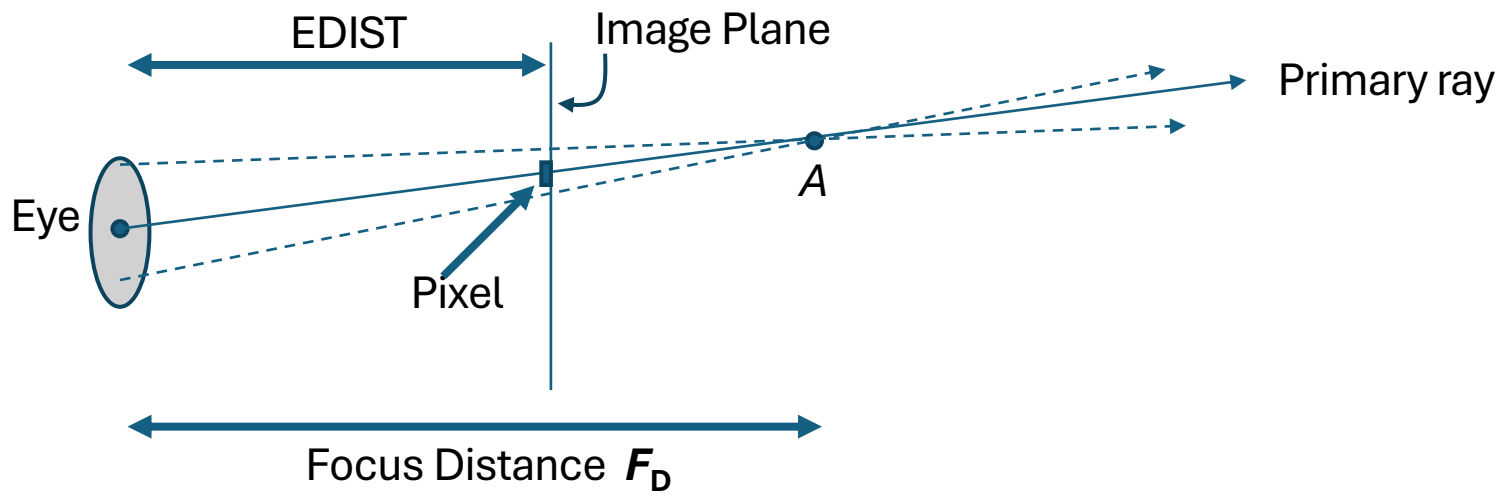
- DoF in Photography



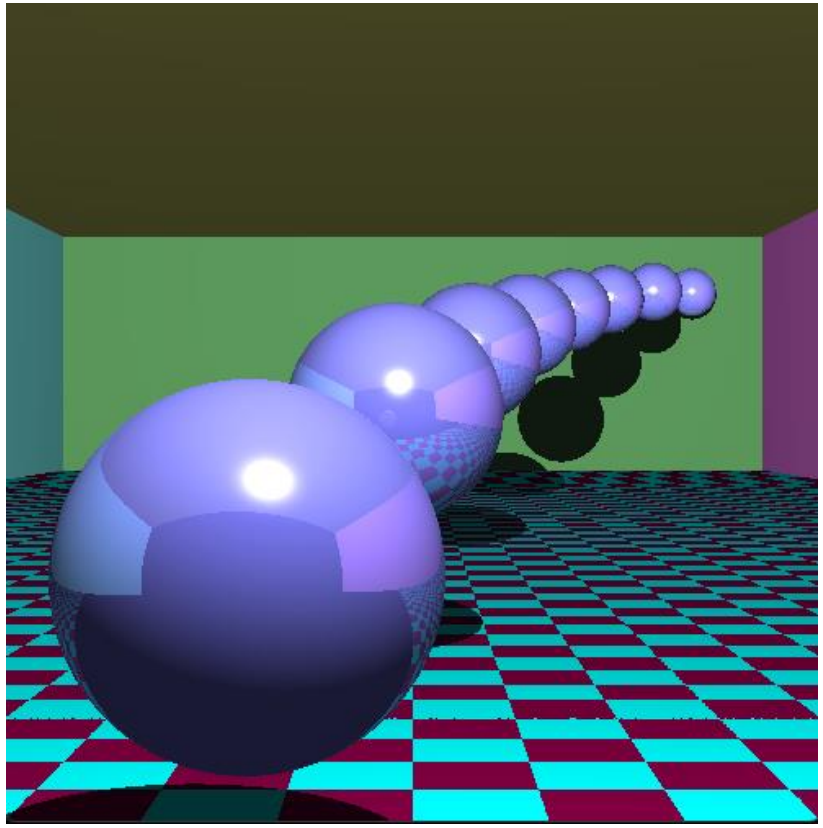
https://capturetheatlas.com/depth-of-field-photography/

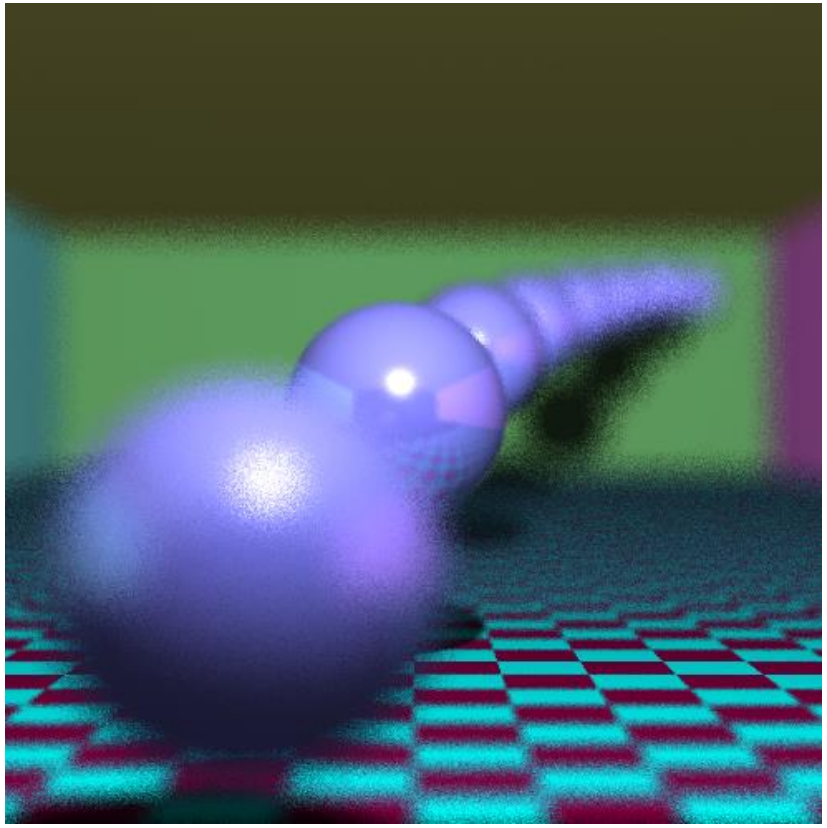
Depth of Field

- DoF in ray tracing is modelled using a “lens” of finite area as the origin of primary rays.
- Instead of using a pin-hole camera model where only a single primary ray is traced through a pixel, we trace multiple rays (randomly generated) from a small area surrounding the eye position, with all rays converging at a point *A* on the primary ray at distance F_D from the eye position. The pixel is assigned the average colour value.



Depth of Field

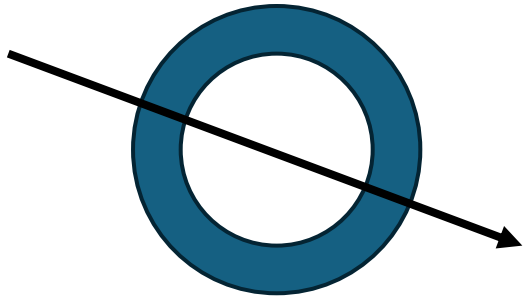


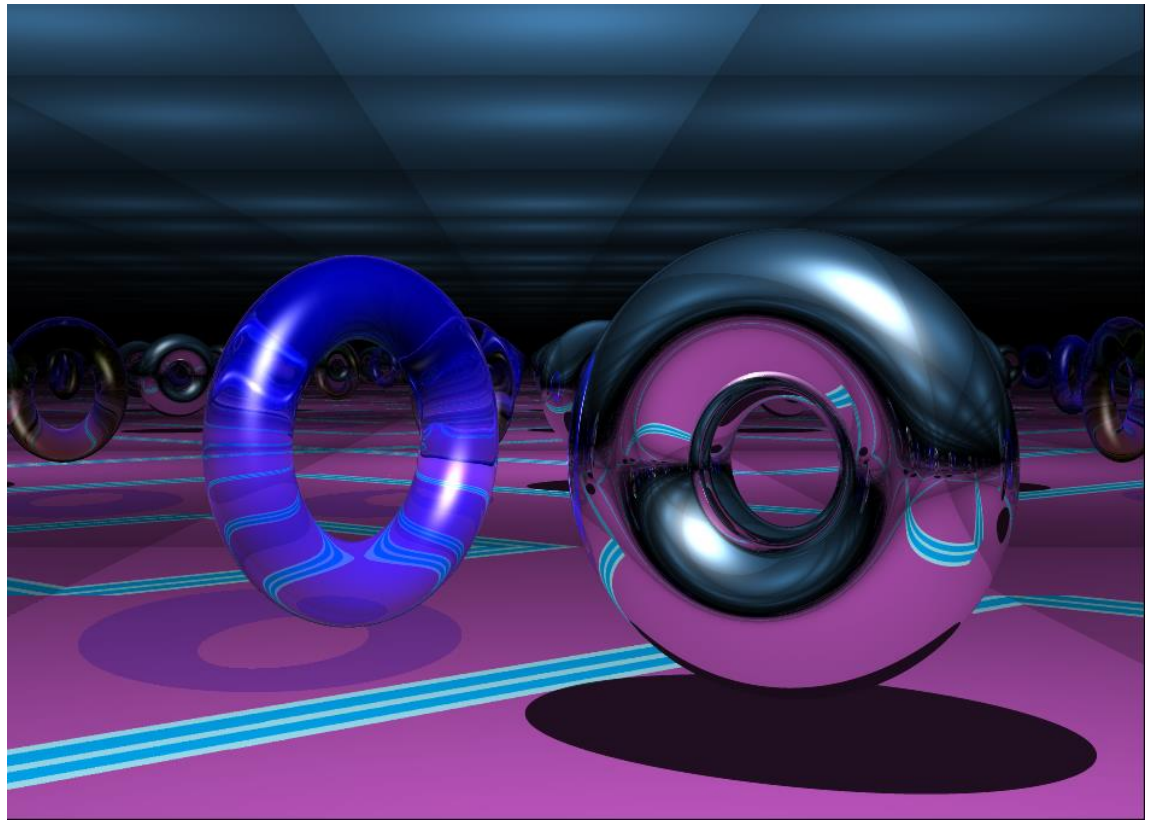


Focus Distance = 200

Torus

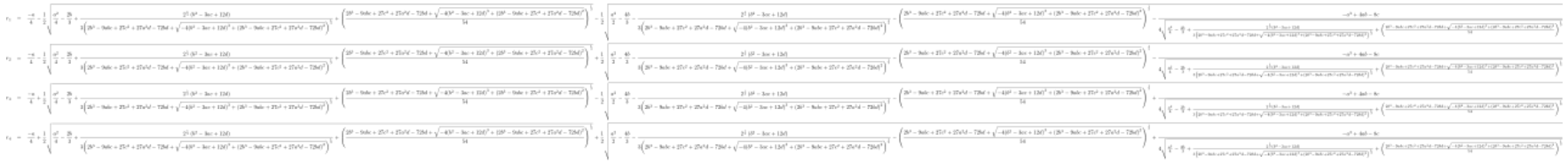
- Up to 4 possible t values
  - Quartic equation
- Can be solved or approximated





Torus Intersection

- Solving the ray-torus intersection is difficult
  - Quartic formula: (Wikipedia)



- Much simpler to approximate it with an iterative method
  - Approximate using Durand Kerner method (Wikipedia)
  - Torus-ray intersection (rearranged into a quartic) and normal equations can be found at http://cosinekitty.com/raytrace/chapter13_torus.html
  - Use doubles for these calculations to avoid visual artifacts

Torus Intersection (cont.)

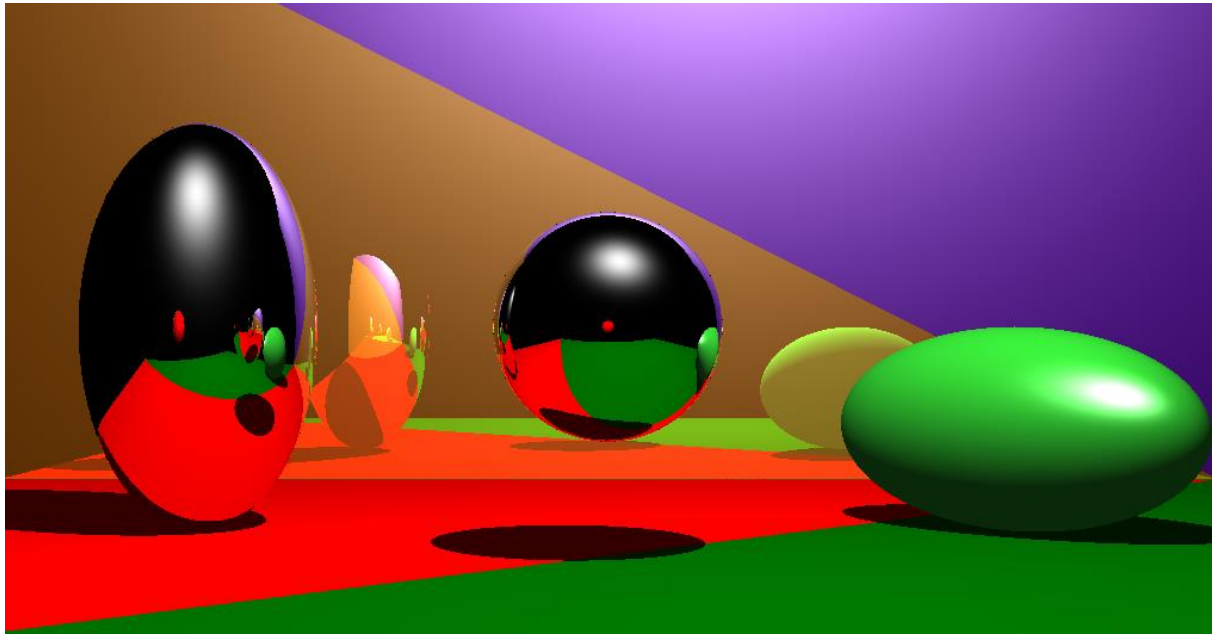
- Durand Kerner method uses complex numbers
  - C++ has a library for complex types and arithmetic

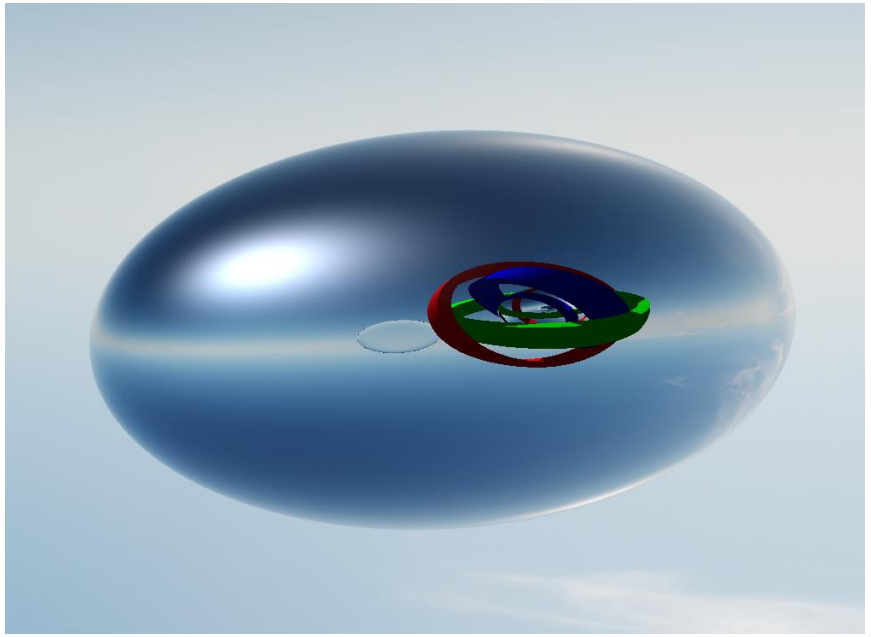
```
#include <complex>
...
std::complex<double> x(1.0, 0.0);
```

- It is simpler to implement a torus that cannot rotate, which is situated at the origin (0, 0, 0)
  - Can use matrix transformations to move and rotate it (object-space transforms)
  - A simpler solution is to show the torus (at the origin) using a mirror or by changing the eye coordinates
- Bounding volume can help with runtime
  - Sphere or bounding box

Transformations

- The effect of the transformations needs to be visually evident
  - Must cause some change that is not possible otherwise
  - E.g. rotate a cylinder or cone off the y-axis, squish a sphere
  - Remember to `#include <glm/gtc/matrix_transform.hpp>`



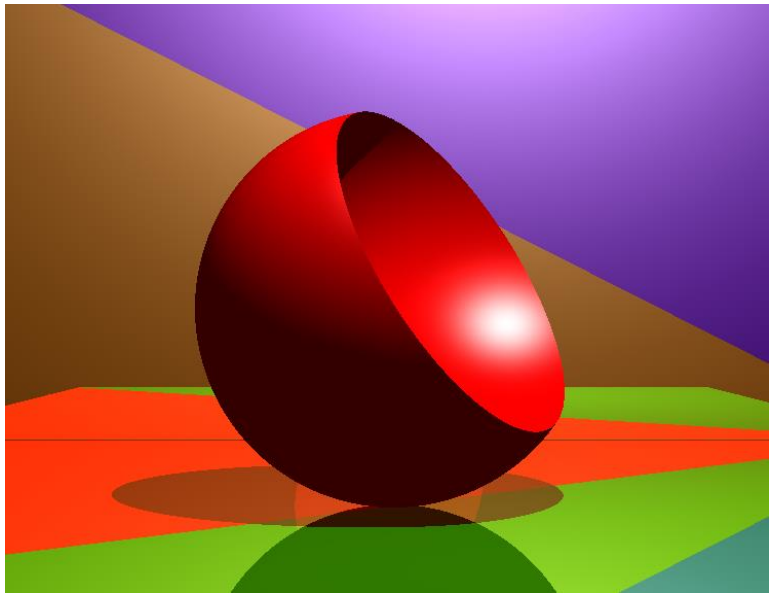


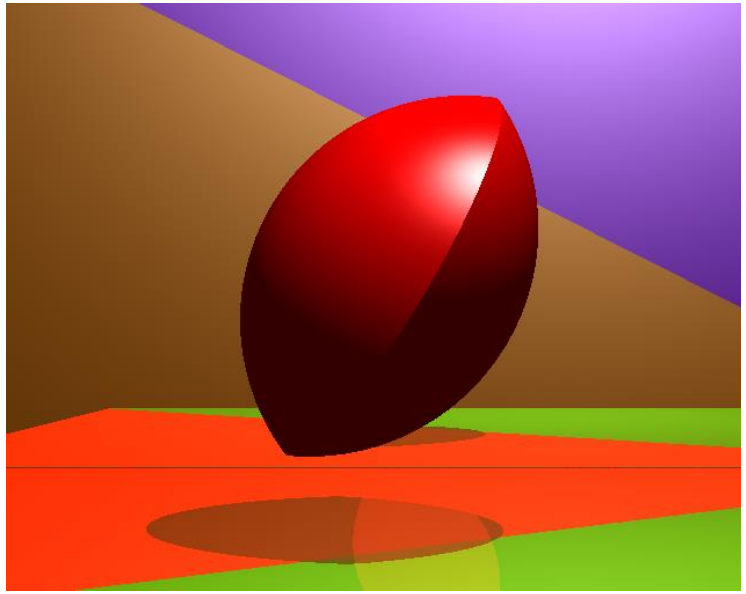
Constructive Solid Geometry (CSG)

- Quite involved to implement (but rewarding!)
- Edit intersect function signatures
  - SceneObject, Sphere, etc.
  - `return std::vector<float>`
- Add a Hit data structure
  - Class or struct
  - Contains t (float), objectIndex (int)
- Re-implement Ray.closestPt()
  - Store a Hit for each point of intersection from each object (std::vector<Hit>) including negative t-values
  - Sort the hits by t-value
  - Use state machine logic to find the first non-negative critical state transition (in or out of a CSG) **or** normal (non-CSG) point of intersection
- Implement logic to flip normals when necessary

Constructive Solid Geometry

- Simple shapes (2 primitives) recommended first
- Must include intersection or subtraction
  - Union-only would be indistinguishable from multiple, non CSG primitives





Assignment Submission

- Provide build details/command in the report
- Please submit report in PDF format only
- Please package all files in a zip folder.